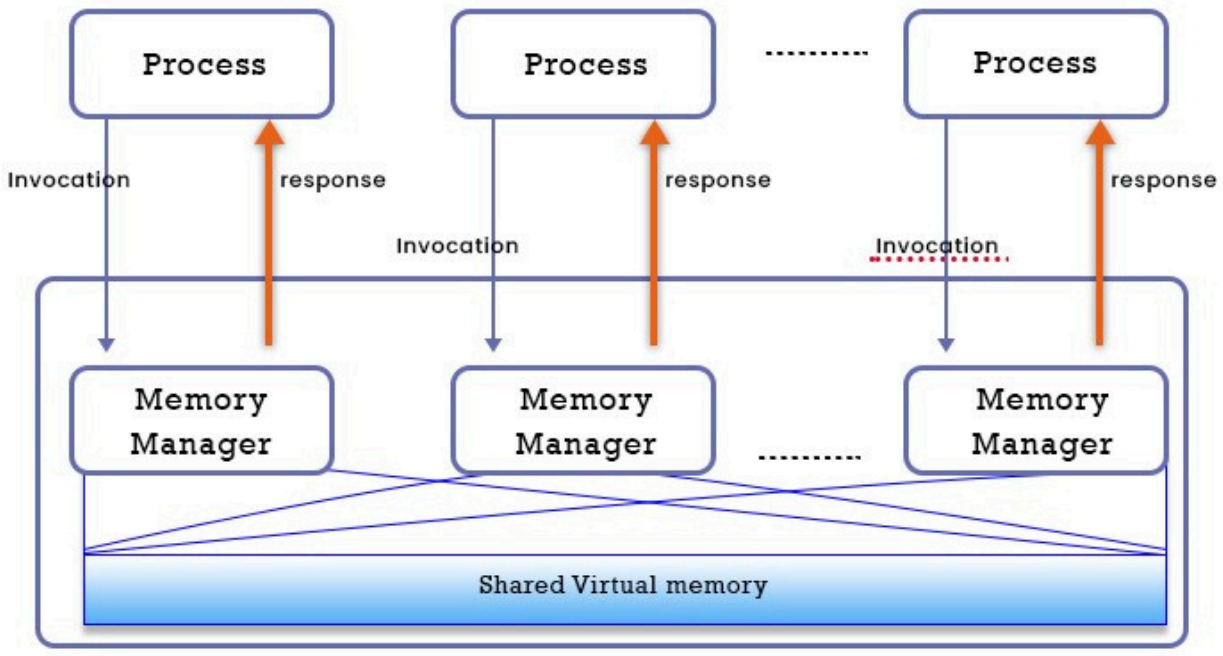
Experiment No: 10

Aim: Distributed shared Memory

Theory:

A distributed shared memory (DSM) system is a collection of many nodes/computers which are connected through some network and all have their local memories. The DSM system manages the memory across all the nodes. All the nodes/computers transparently interconnect and process. The DSM also makes sure that all the nodes are accessing the virtual memory independently without any interference from other nodes. The DSM does not have any physical memory, instead, avirtual space address is shared among all the nodes, and the transfer of data occurs among them through the main memory.



Distributed shared memory

Types of Distributed shared memory

1. On-Chip Memory:
    1. All the data are stored in the CPU's chip.
    2. There is a direct connection between Memory and address lines.
    3. The On-Chip Memory DSM is very costly and complicated.
2. Bus-Based Microprocessor
    1. The connection between the memory and the CPU is established through a number of parallel wires that acts as a bus.

2. All the computer follows some protocols to access the memory, and an algorithm is implemented to prevent memory access by the systems at the same time.
3. The network traffic is reduced by the cache memory.

3. Ring-basedMicroprocessor
    1. In Ring-based DSM, there is no centralized memory present
    2. All the nodes are connected through some link/network, and accessing of memory is done by the token passing
    3. All the nodes/computers present in the shared area access a single address line in this system

Let us now see some advantages of Distributed Shared Memory system:

1. Easy Abstraction - Since the address space is the same, data migration is not an issue for programmers, making it simpler to build than RPC.
2. Easier Potability - Sequential to distributed system migration is made easy by the access protocols employed in DSM. Because they make use of a common programming interface, DSM programmes are portable.
3. Locality of data - Data that is being fetched in large blocks, or data that is close to the memory address being fetched, may be needed in the future.
4. Larger memory space - Large virtual memory space is provided, paging operations are decreased, and the total memory size is the sum of the memory sizes of all the nodes.
5. Better Performance - It speeds up the data access in order to increase the performance of the system.
6. Flexible communication Environment - There is no requirement for the sender and receiver to be present, thus they can join and leave the DSM system without influencing the others.
7. Simplified process Migration - Since they all share the same address space, moving one process to a different computer is simple

Some more basic advantages of the distributed shared memory (DSM) system are listed below:

1. It is less expensive than using multiprocessing systems
2. Data access is done smoothly
3. It provides better scalability as several nodes can access the memory.

Now let's see the disadvantages of the distributed shared memory:

1. Accessing is faster in a non-distributed shared memory systemthan in a distributed system. 2. Simultaneous access to data has always been a topic of discussion. It should ensure some additional protection to it.
3. It generates some programmer control over the actual message
4. In order to write correct programs, programmers must learn the consistency model

```
import threading

# Shared Memory
memory = {}

# Lock for synchronization
lock = threading.Lock()

# Function to set values in shared memory
def set_value(key, value):
    global memory
    lock.acquire()
    memory[key] = value
    lock.release()

# Function to get values from shared memory
def get_value(key):
    global memory
    lock.acquire()
    value = memory.get(key, None)
    lock.release()
    return value

# Function for thread 1
def thread_1():
    set_value("a", 25)
    set_value("b", 18)
    print("Thread 1 sets value of a as 25 and b as 18")

# Function for thread 2
def thread_2():
    value_a = get_value("a")
    value_b = get_value("b")
    print("Thread 2 reads value of a as {} and b as {}".format(value_a, value_b))
    if value_a is not None and value_b is not None:
        set_value("c", value_a * value_b)

# Main Thread
def main_thread():
    thread1 = threading.Thread(target=thread_1)
    thread2 = threading.Thread(target=thread_2)

    thread1.start()
    thread1.join()  # Make sure thread 1 finishes before thread 2 starts

    thread2.start()
    thread2.join()

    value_c = get_value("c")
    print("Main thread reads value of c as {}".format(value_c))

if __name__ == "__main__":
    main_thread()
```

```
Thread 1 sets value of a as 25 and b as 18
Thread 2 reads value of a as 25 and b as 18
Main thread reads value of c as 450
```

Conclusion:

The DSM experiment demonstrates the importance of synchronization in multi-threaded systems. Using locks ensures data consistency and prevents race conditions by controlling access to shared memory, making it crucial for safe interactions in distributed environments.

```
import threading

# Shared Memory (Initialized as a dictionary)
memory = {}

# Lock for synchronization to prevent race conditions
lock = threading.Lock()

# Function to set values in shared memory
def set_value(key, value):
    global memory
    lock.acquire()
    try:
        memory[key] = value
    finally:
        lock.release()

# Function to get values from shared memory
def get_value(key):
    global memory
    lock.acquire()
    try:
        value = memory.get(key, None)
    finally:
        lock.release()
    return value

# Function for Thread 1
def thread_1():
    print("Thread 1: Setting a=25 and b=18")
    set_value("a", 25)
    set_value("b", 18)

# Function for Thread 2
def thread_2():
    value_a = get_value("a")
    value_b = get_value("b")

    print("Thread 2 reads value of a as {} and b as {}".format(value_a, value_b))

    if value_a is not None and value_b is not None:
        result = value_a + value_b
        print(f"Thread 2: Calculating c = a + b = {result}")
        set_value("c", result)

# Main Thread
def main_thread():
    t1 = threading.Thread(target=thread_1)
    t2 = threading.Thread(target=thread_2)

    t1.start()
    t1.join() # Ensures thread 1 finishes before thread 2 starts

    t2.start()
    t2.join()

    value_c = get_value("c")
    print("Main thread reads value of c as {}".forma
t(value_c))

if __name__ == "__main__":
    main_thread()
```